



Windows Socket Programming: UDP, TCP, C



Reti di Calcolatori Modulo 2

Sommario

- ▶ Richiami di TCP
- ▶ Classificazione dei socket
- ▶ Socket client-server UDP
- ▶ Socket client-server TCP
- ▶ Esempio server multiporta
- ▶ gethostbyname & gethostbyaddr
- ▶ Java Socket

TCP/IP: interfaccia di programmazione

► Pila TCP/IP

► Livello Applicazione

► Applicazioni standard

- HTTP
- FTP
- Telnet

► Applicazioni utente

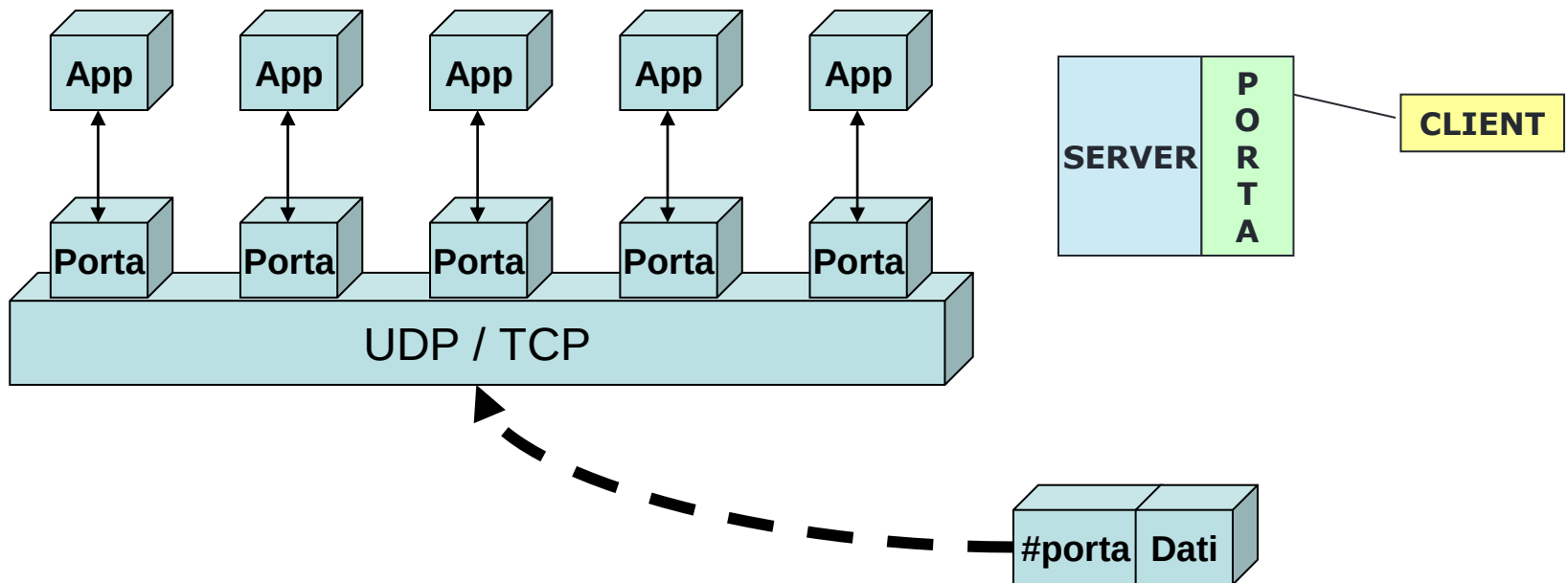
Libreria di funzioni per C e Java:
▪ **SOCKET LIBRARY**

► Livello Trasporto

- TCP
- UDP

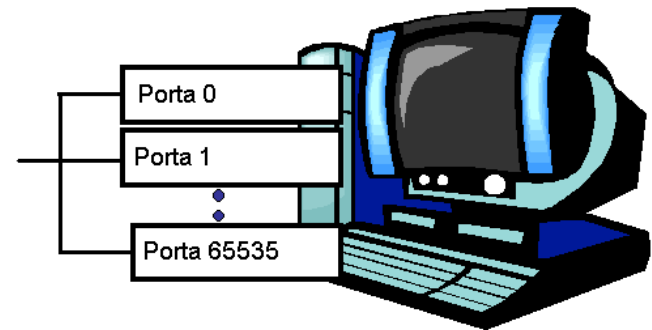
Richiami di TCP: porte

- ▶ I protocolli TCP e UDP usano le porte per mappare i dati in ingresso con un particolare processo attivo su un computer
- ▶ Ogni socket è legato ad un numero di porta così che il livello TCP può identificare l'applicazione a cui i dati devono essere inviati



Richiami di TCP: well-known ports

- ▶ Porte rappresentate da valori interi positivi (16 bit)
- ▶ Rappresentano punto di collegamento fra strato fisico e applicazioni; rappresentano un canale di comunicazione
- ▶ Alcune porte sono state riservate per il supporto di servizi well-known
 - ▶ ftp -> 21/tcp
 - ▶ telnet -> 23/tcp
 - ▶ smtp -> 25/tcp
 - ▶ login -> 513/tcp
- ▶ I servizi e i processi a livello utente generalmente usano un numero di porta ≥ 1024



Socket: Definizione

- ▶ A livello programmatico, un Socket è definito come un “identificativo univoco che rappresenta un canale di comunicazione attraverso cui l’informazione è trasmessa” [RFC 147]
- ▶ La comunicazione basata su socket è indipendente dal linguaggio di programmazione
 - ▶ Un programma socket scritto in Java può comunicare con un programma non Java
- ▶ Client e server devono concordare solo su protocollo (TCP o UDP) e numero di porta

Socket: Definizione

- ▶ I socket forniscono un'interfaccia per la programmazione dell'accesso alla rete al livello di trasporto
 - ▶ Un socket è un punto finale di un collegamento comunicativo bidirezionale tra due programmi in esecuzione sulla rete
 - ▶ Un socket è legato a un numero di porta così che il livello TCP può identificare l'applicazione a cui i dati devono essere inviati
- ▶ Usando i socket la comunicazione in rete è molto simile all'I/O su file
 - ▶ Gestione dei socket trattata come la gestione di file
 - ▶ I comandi di lettura e scrittura per l'I/O su file sono applicabili anche all'I/O su socket

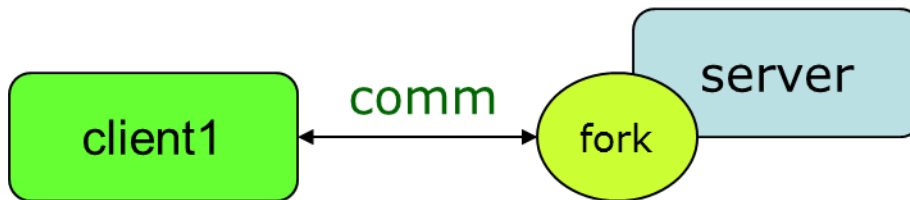
Socket: Comunicazione

- ▶ Un server (programma) gira su un computer specifico e ha un socket legato a una porta specifica
- ▶ Un server aspetta e ascolta sul socket finché un client non esegue una richiesta di connessione
- ▶ Se tutto funziona, un server accetta la connessione e ottiene dal sistema operativo locale un nuovo socket
- ▶ In questo modo un server può continuare ad ascoltare sul socket originale per eventuali altre richieste di connessione, mentre comunica con il client già connesso

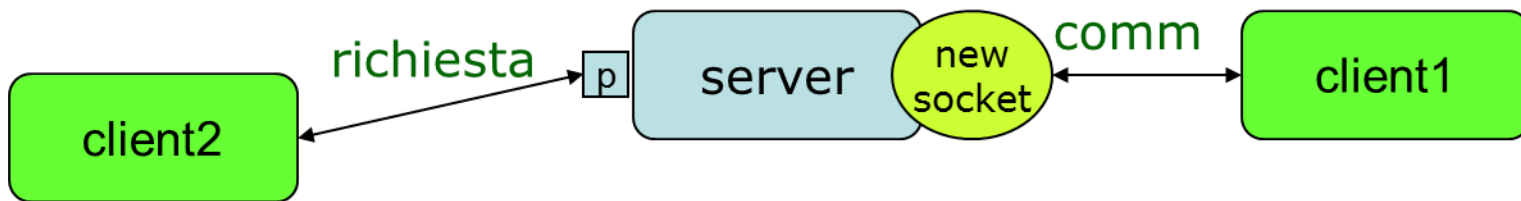
Socket: Comunicazione



Il server riceve
una richiesta da
un client



Crea un nuovo socket
che dialogherà col client
e torna in ascolto di
altre richieste



Indirizzi, porte e socket

La metafora della posta

- ▶ L'utente è l'applicazione
- ▶ Il suo indirizzo di casa è l'indirizzo
- ▶ La casella postale è la porta
- ▶ L'ufficio postale è la rete
- ▶ Il socket è la chiave che dà all'utente l'accesso alla casella postale giusta
 - ▶ **NOTA BENE:** si presuppone che anche la posta in uscita venga posta dall'utente nella sua casella postale invece di essere imbucata in una buca delle lettere



Socket: Classificazione

▶ Datagram Socket

- ▶ Utilizzano un protocollo **senza connessione** -> UDP
- ▶ Non è necessaria una procedura iniziale di connessione e riconoscimento fra client e server
- ▶ Non dà garanzie di ricezione ed ordine dei pacchetti

▶ Stream Socket

- ▶ Utilizzano un protocollo **con connessione** -> TCP
- ▶ La comunicazione avviene solo fra nodi tra cui è stato stabilito un canale di comunicazione
- ▶ Garantisce ordine e ricezione dei pacchetti

▶ Raw socket

- ▶ Dà completo accesso allo strato fisico del sistema per poter sfruttare funzionalità non implementate dalle interfacce
- ▶ Solo per utenti esperti
- ▶ Utilizzano protocolli di connessione IP

Socket: Classificazione

▶ Sincroni

- ▶ il client ed il server si sincronizzano ad ogni messaggio: sia `send()` (per inviare) che `recv()` (per ricevere) sono bloccanti
- ▶ `send()` ritorna dopo che è stata fatto l'invio del messaggio
- ▶ `recv()` ritorna solo dopo che il messaggio arriva in coda

▶ Asincrona

- ▶ `send()` non bloccante significa che l'operazione ritorna subito dopo che il messaggio è stato copiato su un buffer locale
- ▶ `recv()` non bloccante significa che ritorna subito il controllo all'applicazione, poi tramite polling o interrupt notificherà l'accodamento del messaggio nella coda

Win Socket C



Differenze ambiente Unix e Windows

- ▶ Include file, tutte le librerie raccolte in una:

```
#include <sys/types.h>  
#include <sys/socket.h>  
#include <netinet/in.h>  
#include <arpa/inet.h>  
#include <netdb.h>
```



```
#include <winsock.h>
```

- ▶ *WSAGetLastError()* al posto della variabile *errno*
- ▶ *closeSocket()* al posto di *close()*
- ▶ Non supporta *Raw Socket*
- ▶ *Fork* necessita di *funzioni alternative*

↳ Funziona solo in ambienti Unix

UDP/TCP client: Header

```
#include <stdio.h>      /* for printf(), fprintf() */
#include <winsock.h>     /* for socket(),... */
#include <stdlib.h>      /* for exit() */

#define ECHOMAX 255     /* Longest string to echo */

void DieWithError(char *errorMessage); /* External error handling function */

void main(int argc, char *argv[])
{
    int sock;                /* Socket descriptor */
    struct sockaddr_in echoServAddr; /* Echo server address */
    struct sockaddr_in fromAddr;    /* Source address of echo */
    unsigned short echoServPort;    /* Echo server port */
    unsigned int fromSize;          /* In-out of address size for recvfrom() */
    char *servIP;                 /* IP address of server */
    char *echoString;             /* String to send to echo server */
    char echoBuffer[ECHOMAX];     /* Buffer for echo string */
    int echoStringLen;            /* Length of string to echo */
    int respStringLen;            /* Length of response string */
    WSADATA wsaData;              /* Structure for WinSock setup communication */
```

Struct sockaddr generica

- ▶ struct sockaddr {
 - u_short sa_family;
 - char sa_data[14];};
- ▶ sa_family
 - ▶ Specifica quale famiglia di indirizzi verrà usata.
 - ▶ Determina come vengono usati i restanti 14 byte.

Struct sockaddr specifica per IP

- ▶ **struct sockaddr_in** {
 short sin_family;
 u_short sin_port;
 struct in_addr sin_addr;
 char sin_zero[8];
};
- ▶ sin_family: AF_INET
- ▶ sin_port: numero porta (0-65.535)
- ▶ sin_addr: indirizzo IP
- ▶ sin_zero: inutilizzato

Ordinamento dei byte di indirizzi e porte

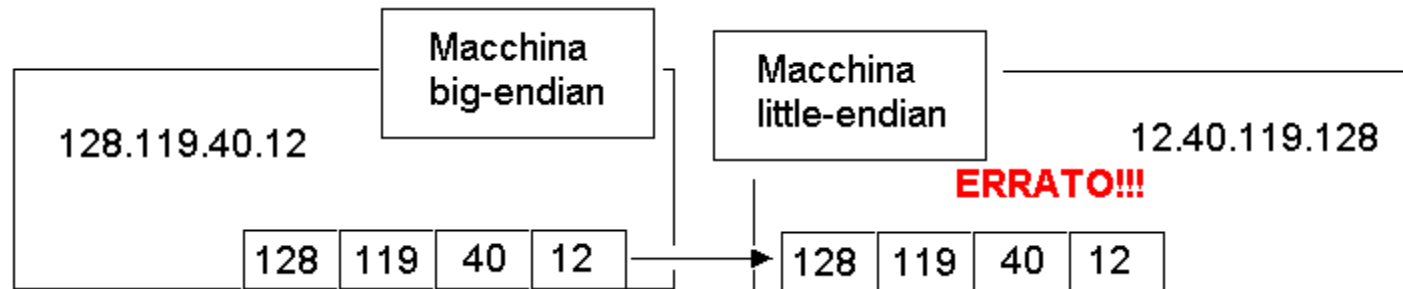
- ▶ Indirizzo e porta memorizzati come interi
 - `u_short sin_port;` (16 bit)
 - `in_addr sin_addr;` (32 bit)
- ▶ Macchine e sistemi operativi diversi usano ordinamenti di parole differenti
 - Little-endian: prima i byte meno significativi
 - Big-endian: prima i byte più significativi

Come possono comunicare sulla rete?

Ordinamento dei byte di indirizzi e porte

- ▶ Indirizzo e porta memorizzati come interi
 - `u_short sin_port;` (16 bit)
 - `in_addr sin_addr;` (32 bit)
- ▶ Macchine e sistemi operativi diversi usano ordinamenti di parole differenti
 - Little-endian: prima i byte meno significativi
 - Big-endian: prima i byte più significativi

Come possono comunicare sulla rete?

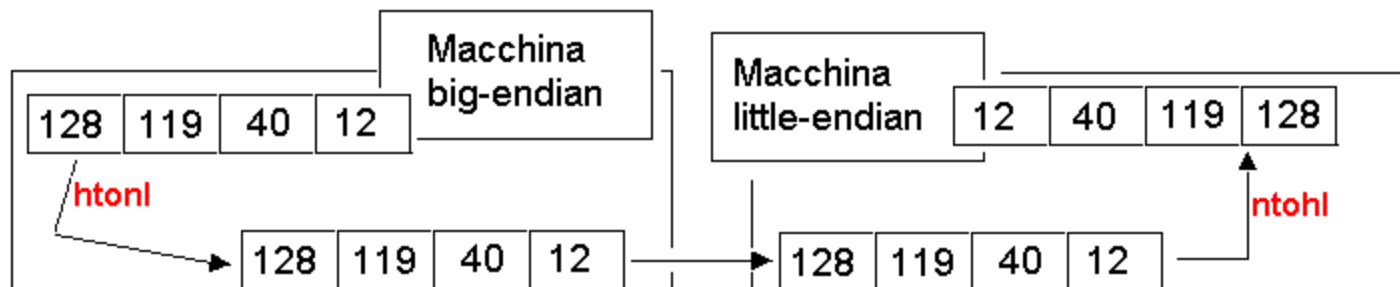


Ordinamento dei byte

- ▶ Ordinamento dei byte usato da un host
 - ▶ Big endian o little-endian
- ▶ Ordinamento dei byte usato dalla rete
 - ▶ Sempre big-endian
- ▶ Qualsiasi parola inviata attraverso la rete deve essere convertita nell'ordine dei byte di rete prima della trasmissione
- ▶ Una volta ricevuta deve essere riconvertita nell'ordine dei byte dell'host

Funzioni di ordinamento dei byte UNIX

- ▶ `u_long htonl(u_long x);`
 - ▶ `u_short htons(u_short x);`
 - ▶ `u_long ntohl(u_long x);`
 - ▶ `u_short ntohs(u_short x);`
- ▶ Sulle macchine big-endian, queste routine non fanno nulla
 - ▶ Sulle macchine little-endian, invertono l'ordinamento dei byte



- ▶ Lo stesso codice funziona indipendentemente dal tipo di endian delle due macchine

UDP/TCP Client: Inizializzazione librerie

- ▶ `WSAStartup(version, WSADATA)`
 - ▶ WORD version, versione delle librerie da caricare
 - ▶ `LPWSADATA WSADATA`, struttura per ricevere i dettagli della Windows Socket Implementation
- ▶ Necessaria solo in ambienti Windows

```
if (WSAStartup(MAKEWORD(2, 0), &wsaData) != 0) /* Load Winsock 2.0 DLL */
{
    fprintf(stderr, "WSAStartup() failed");
    exit(1);
}
```

UDP/TCP Client: Creazione del socket

- ▶ `socket(af, type, protocol)`
 - ▶ `int af`, famiglia di indirizzi da utilizzare (`AF_INET`, `PF_INET`, `AF_UNIX`)
 - ▶ `int type`, tipo di socket (`SOCK_STREAM`, `SOCK_DGRAM`)
 - ▶ `int protocol`, tipo di protocollo utilizzato (`IPPROTO_TCP`, `IPPROTO_UDP`)
- ▶ Restituisce un valore negativo se non va a buon fine

```
/* Create a best-effort datagram socket using UDP */  
if ((sock = socket(PF_INET, SOCK_DGRAM, IPPROTO_UDP)) < 0)  
    DieWithError("socket() failed");
```

UDP Client: sendTo()

- ▶ sendTo(socket, echoString, bufferLength, flag, to, toLength)
 - ▶ int socket, descrittore del socket
 - ▶ char* echoString, messaggio da spedire
 - ▶ int bufferLength, dimensione in bytes
 - ▶ int flags, indica la modalità con cui viene effettuata la chiamata (default 0)
 - ▶ sockaddr* to, indirizzo destinatario
 - ▶ int toLength, dimensione indirizzo in byte
- ▶ Non è necessario una procedura di connessione fra client e server

```
/* Construct the server address structure */
memset(&echoServAddr, 0, sizeof(echoServAddr)); /* Zero out structure */
echoServAddr.sin_family = AF_INET; /* Internet address family */
echoServAddr.sin_addr.s_addr = inet_addr(servIP); /* Server IP address */
echoServAddr.sin_port = htons(echoServPort); /* Server port */

/* Send the string, including the null terminator, to the server */
if (sendto(sock, echoString, echoStringLen, 0, (struct sockaddr *)
    &echoServAddr, sizeof(echoServAddr)) != echoStringLen)
    DieWithError("sendto() sent a different number of bytes than expected");
```


UDP Client: recvfrom()

- ▶ `recvfrom(socket, buffer, bufferLength, flags, from, fromLength)`
 - ▶ `SOCKET` socket, descrittore del socket
 - ▶ `char *buffer`, messaggio spedito
 - ▶ `int bufferLength`, dimensione in bytes
 - ▶ `int flags`, indica la modalità con cui viene effettuata la chiamata (default 0)
 - ▶ `sockaddr* from`, indirizzo mittente
 - ▶ `int fromLength`, dimensione indirizzo in byte

```
/* Receive a response */
```

```
fromSize = sizeof(fromAddr);  
if ((respStringLen = recvfrom(sock, echoBuffer, ECHOMAX, 0, (struct sockaddr *) &fromAddr,  
                             &fromSize)) != echoStringLen)  
    DieWithError("recvfrom() failed");
```

UDP Client: Chiusura socket

- ▶ `closesocket(socket)`: chiude il socket specificato e ritorna 0 se la chiusura è andata a buon fine, altrimenti un codice d'errore, recuperabile con `WSAGetLastError()`
- ▶ `WSACleanup()`: rilascia la libreria

```
closesocket(sock);  
WSACleanup(); /* Cleanup Winsock */  
  
exit(0);  
}
```

UDP Server: Port Binding

- ▶ `bind(socket, address, addressLength)`
 - ▶ `SOCKET socket`, descrittore del socket
 - ▶ `sockaddr* address`, indirizzo del socket
 - ▶ `int addressLength`, dimensione dell'indirizzo in byte
- ▶ Lega ogni comunicazione su quel protocollo e su quella porta all'applicazione

```
/* Construct local address structure */
memset(&echoServAddr, 0, sizeof(echoServAddr)); /* Zero out structure */
echoServAddr.sin_family = AF_INET; /* Internet address family */
echoServAddr.sin_addr.s_addr = htonl(INADDR_ANY); /* Any incoming interface */
echoServAddr.sin_port = htons(echoServPort); /* Local port */

/* Bind to the local address */
if (bind(sock, (struct sockaddr *) &echoServAddr, sizeof(echoServAddr)) < 0)
    DieWithError("bind() failed");
```

UDP Server: Ricezione messaggi

- ▶ Il server rimane in attesa della ricezione di un messaggio sulla `recvfrom()`
- ▶ Nella *struct echoClntAddr* sono salvate tutte le informazioni relative al chiamante (in *sin_addr* l'indirizzo IP)

```
for (;;) /* Run forever */ → Iterativo
{
    /* Set the size of the in-out parameter */
    cliLen = sizeof(echoClntAddr);

    /* Block until receive message from a client */
    if ((recvMsgSize = recvfrom(sock, echoBuffer, ECHOMAX, 0,
        (struct sockaddr *) &echoClntAddr, &cliLen)) < 0)
        DieWithError("recvfrom() failed");

    printf("Handling client %s\n", inet_ntoa(echoClntAddr.sin_addr));

    /* Send received datagram back to the client */
    if (sendto(sock, echoBuffer, recvMsgSize, 0, (struct sockaddr *) &echoClntAddr,
        sizeof(echoClntAddr)) != recvMsgSize)
        DieWithError("sendto() sent a different number of bytes than expected");
}
```

UDP Socket

```
...
if (WSAStartup(MAKEWORD(2, 0), &wsaData) != 0) /* Load Winsock 2.0 DLL */
/* Create a best-effort datagram socket using UDP */
if ((sock = socket(PF_INET, SOCK_DGRAM, IPPROTO_UDP)) < 0)
...
memset(&echoServAddr, 0, sizeof(echoServAddr)); /* Zero out structure */
echoServAddr.sin_family = AF_INET; /* Internet address family */
echoServAddr.sin_addr.s_addr = inet_addr(...);
echoServAddr.sin_port = htons(echoServPort); /* Server port */
```

Server: INADDR_ANY
Client: servIP

```
/* Send the string, including the null terminator, to the server */
if (sendto(sock, echoString, echoStringLen, 0, (struct sockaddr *)
...
if ((respStringLen = recvfrom(sock, echoBuffer, ECHOMAX, 0, (struct sockaddr *) &fromAddr,
    &fromSize)) != echoStringLen)
...
closesocket(sock);
WSACleanup(); /* Cleanup Winsock */
```

Client

Server

```
if (bind(sock, (struct sockaddr *) &echoServAddr, sizeof(echoServAddr)) < 0)
    DieWithError("bind() failed");

for (;;) /* Run forever */
{
    /* Block until receive message from a client */
    if ((recvMsgSize = recvfrom(sock, echoBuffer, ECHOMAX, 0,
        (struct sockaddr *) &echoClntAddr, &cliLen)) < 0)
        /* Send received datagram back to the client */
        if (sendto(sock, echoBuffer, recvMsgSize, 0, (struct sockaddr *) &echoClntAddr,
            sizeof(echoClntAddr)) != recvMsgSize)
    }
}
```

TCP Client: Creazione del socket

- ▶ Crea un socket basato su TCP di tipo STREAM

```
/* Create a reliable, stream socket using TCP */  
if ((sock = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP)) < 0)  
    DieWithError("socket() failed");
```

TCP Client: Connessione

- ▶ Stabilisce una connessione verso uno specifico server
- ▶ `connect(socket, address, addressLength)`
 - ▶ `SOCKET socket`, descrittore del socket
 - ▶ `sockaddr* address`, indirizzo del server
 - ▶ `int addressLength`, dimensione dell'indirizzo in byte

```
/* Construct the server address structure */
memset(&echoServAddr, 0, sizeof(echoServAddr)); /* Zero out structure */
echoServAddr.sin_family      = AF_INET;          /* Internet address family */
echoServAddr.sin_addr.s_addr = inet_addr(servIP); /* Server IP address */
echoServAddr.sin_port        = htons(echoServPort); /* Server port */
/* Establish the connection to the echo server */
if (connect(sock, (struct sockaddr *) &echoServAddr, sizeof(echoServAddr)) < 0)
    DieWithError("connect() failed");
```

TCP Client: Invio dati

- ▶ Invia i dati specificati verso un socket già connesso
- ▶ Non è necessario indicare ogni volta l'indirizzo del destinatario
- ▶ `send(socket, buffer, bufferLength, flags)`
 - ▶ SOCKET socket, descrittore del socket
 - ▶ `char* buffer`, messaggio da inviare
 - ▶ `int bufferLength`, dimensione del messaggio in byte
 - ▶ `int flag`, indica la modalità con cui viene effettuata la chiamata (default 0)

```
/* Send the string, including the null terminator, to the server */  
if (send(sock, echoString, echoStringLen, 0) != echoStringLen)  
    DieWithError("send() sent a different number of bytes than expected");
```


TCP Client: Ricezione dati

- ▶ Riceve dati da un socket connesso e restituisce il numero di byte ricevuti
- ▶ Quando restituisce 0 la comunicazione è interrotta
- ▶ `recv(socket, buffer, bufferLength, flag)`
 - ▶ `SOCKET socket`, descrittore del socket
 - ▶ `char* buffer`, zona di memoria del messaggio da ricevere
 - ▶ `int bufferLength`, dimensione del messaggio in byte
 - ▶ `int flag`, indica la modalità con cui viene effettuata la chiamata (default 0)

```
/* Receive up to the buffer size (minus 1 to leave space for  
a null terminator) bytes from the sender */  
if ((bytesRcvd = recv(sock, echoBuffer, RCVBUFSIZE - 1, 0)) <= 0)  
    DieWithError("recv() failed or connection closed prematurely");
```

TCP Server: Connessione

- ▶ Dopo il bind, il server rimane in ascolto (listen) sul socket
- ▶ `listen(socket, backlog)`
 - ▶ SOCKET socket, descrittore del socket
 - ▶ int backlog, dimensione massima della coda delle connessioni in attesa

```
/* Bind to the local address */
if (bind(servSock, (struct sockaddr *) &echoServAddr, sizeof(echoServAddr)) < 0)
    DieWithError("bind() failed");

/* Mark the socket so it will listen for incoming connections */
if (listen(servSock, MAXPENDING) < 0)
    DieWithError("listen() failed");
```

TCP Server: Accettazione

- ▶ Quando un client richiede la connessione, viene creato un nuovo socket per gestire la comunicazione
- ▶ Il socket iniziale rimane in attesa di nuove richieste
- ▶ `accept(socket, clientAddress, clientAddressLength)`
 - ▶ `SOCKET socket`, descrittore del socket
 - ▶ `sockaddr* clientAddress`, indirizzo del client che ha richiesto l'accesso
 - ▶ `int clientAddressLength`, dimensione di `clientAddress` in byte

```
/* Wait for a client to connect */  
if ((clntSock = accept(servSock, (struct sockaddr *) &echoClntAddr, &clntLen)) < 0)  
    DieWithError("accept() failed");
```

Socket Select

- ▶ Primo passo: creazione di un array di socket ognuno su una porta differente; nell'esempio la lista delle porte è passata a riga di comando
- ▶ Per ogni porta è creato un socket server distinto

```
/* Create list of ports and sockets to handle ports */
for (port = 0; port < noPorts; port++)
{
    /* Add port to port list */
    portNo = atoi(argv[port + 2]); /* Skip first two arguments */

    /* Create port socket */
    servSock[port] = CreateTCPServerSocket(portNo);
}
```

Socket Select

- ▶ Inizializzare l'insieme dei socket marcabili
 - ▶ `FD_ZERO()` svuota l'insieme dei socket marcati
 - ▶ `FD_SET()`, prepara la lista dei socket attivi, aggiungendo all'insieme il socket che potrà poi essere marcato dalla `select()`

```
FD_ZERO(&sockSet);  
for (port = 0; port < noPorts; port++)  
    FD_SET(servSock[port], &sockSet);
```

Socket Select

- ▶ La select() ad intervalli specificati, controlla quali socket hanno fatto richiesta in lettura, scrittura o hanno riscontrato errori di comunicazione e restituisce il numero di socket interessati
- ▶ I socket richiedenti sono marcati ed inseriti in specifici array per le successive operazioni
- ▶ select(0,readfds, writefds, errorfds, timeout):
 - ▶ fd_set* readfs, puntatore all'array di socket in lettura;
 - ▶ fd_set* writefs, puntatore all'array di socket in scrittura;
 - ▶ fd_set* errorfs, puntatore all'array di socket controllati per errori;
 - ▶ timeval* timeout, tempo di attesa per la select

```
/* Suspend program until descriptor is ready or timeout */  
/* The first parameter to select() is ignored in Winsock */  
if (select(0, &sockSet, NULL, NULL, &selTimeout) == 0)  
    printf("No echo requests for %ld secs...Server still alive\n", timeout);
```

Socket Select

- ▶ Per capire quali socket hanno richiesto l'accesso, si scorre l'array dei socket e si applica `FD_ISSET()`
- ▶ `FD_ISSET` restituisce un valore non-zero se il socket è stato marcato
- ▶ `FD_ISSET(socket, *set):`
 - ▶ `SOCKET` socket, il descrittore del socket da controllare
 - ▶ `fd_set *set`, puntatore all'insieme dei socket marcati

```
for (port = 0; port < noPorts; port++)  
    if (FD_ISSET(servSock[port], &sockSet))  
    {  
        printf("Request on port %d (cmd-line position): ", port);  
        HandleTCPClient(AcceptTCPConnection(servSock[port]));  
    }
```

